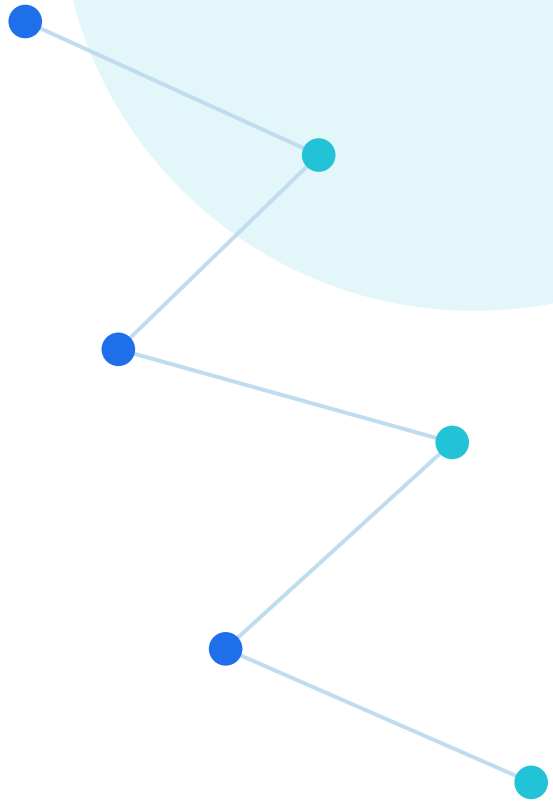
PS-033: DNS-Based Discovery of Diagnostic Services

AIORI Hackathon 3

Cloud Computing & IoT



Team Learner

Debjit Bera – debjitbera365@gmail.com

Arpan Sardar – arpansardar106@gmail.com

Sayani Manna (Faculty mentor) – sayaniresearch2018@gmail.com

Presentation Date: 12/10/2026

Abstract and Problem Description

Problem

Clients must find lab diagnostic services (latency, traceroute, connectivity) without static IPs or a vendor registry [11].

Why it matters

Moved endpoints silently break monitoring; forged or malformed discovery data can misdirect clients.

Approach

Publish DNS-AID-style SVCB metadata in a DNSSEC-signed test zone; one client queries, validates, parses, then invokes with credentials [1].

Intended outcome

A lab prototype: 3 mock services, 1 client, and tests for valid, bogus, malformed and unavailable cases. No Internet-wide registry.

Hardcoded IP addresses

- Endpoint moves: every client config must be edited
- Stale entries fail silently or reach the wrong host
- No machine-readable protocol or capability data

Proprietary registries

- Extra vendor system to run, trust and keep available
- Discovery data lives outside DNS (lock-in)
- A registry outage can break discovery for all clients

Why dynamic discovery: DNS is already deployed, cached and operated by the zone owner, so moving a service means updating one record instead of every client.

Background

DNS

Simply: The Internet's lookup system that maps names to data [2][3].

Here: Our test zone stores the service records.

DNS-SD and DNS-AID

Simply: A convention for finding services by querying DNS names [9].

Here: The DNS-AID draft publishes endpoints as SVCB records [1].

DNSSEC

Simply: Signatures on DNS data: proves origin and integrity, not secrecy [4].

Here: The client rejects any record that fails validation.

Structured metadata

Simply: Named fields describing a service: endpoint, protocol, capability.

Here: SVCB parameters such as alpn, port, cap, cap-sha256 [1][7].

Mock diagnostic services

Simply: Lab stand-ins for real probes that return simulated data.

Here: Latency, traceroute and connectivity (our proposed examples).

Authenticated invocation

Simply: The caller proves who it is; the service decides what is allowed.

Here: A token check at each service, after discovery.

DNS discovery
Where is the service?

DNSSEC
Is the DNS data genuine?

Metadata validation
Is the metadata well-formed?

Authn / authz
May this caller invoke it?

Literature & RFC Review

Reference	Status	Contribution to our design
draft-mozleywilliams-dnsop-dnsaid-02 [1]	Internet-Draft, 27 May 2026	SVCB records with cap, cap-sha256, well-known; DNSSEC-signed; records are not a trust signal
RFC 1034, RFC 1035 [2][3]	STD 13, 1987	DNS concepts, zones, resource records, message formats
RFC 4033 [4]	Standards Track, 2005	DNSSEC introduction, trust anchors, chain of trust; no confidentiality
RFC 4034 [5]	Standards Track, 2005	DNSKEY, RRSIG, DS and NSEC record types
RFC 4035 [6]	Standards Track, 2005	Validation procedure; AD bit; SERVFAIL for bogus data
RFC 9460 [7]	Standards Track, 2023	SVCB record type that DNS-AID builds on

Established standards

RFCs are published, stable specifications. We rely on them for DNS behavior, DNSSEC validation and the SVCB record type. See also BCP 237 [8].

Internet-Draft

Work in progress, not IETF-endorsed, may change or expire (expires 28 Nov 2026). We pin to revision -02 and implement only a subset.

Design assumption: the draft targets AI-agent discovery; we reuse its SVCB record model for mock diagnostic services.

Proposed Solution

Components: publisher → signed test zone → validating resolver → discovery client → 3 mock services

1. **Discover:** SVCB query for the service name
2. **Validate:** DNSSEC check; reject bogus data
3. **Parse:** pinned SVCB subset, schema, cap-sha256
4. **Authenticate:** call endpoint with a bearer token
5. **Process:** read the service's JSON response
6. **Handle errors:** one typed error code per failure

Pseudocode (design, not yet implemented)

```
svcb = resolve("latency.diag.lab.example", SVCB)
if svcb.rcode == SERVFAIL or not svcb.AD:
    fail("BOGUS_DNS")
meta = parse(svcb, PINNED_PARAMS)
if not schema_ok(meta): fail("MALFORMED_METADATA")
desc = fetch(meta.well_known)
if sha256(canon(desc)) != meta.cap_sha256:
    fail("MALFORMED_METADATA")
r = call(meta.endpoint, token=TOKEN, req)
if r.timeout or r.conn_error: fail("UNAVAILABLE")
if r.status in (401, 403): fail("UNAUTHORIZED")
return r.body
```

Three proposed mock services (examples, not names mandated by PS-033)

Service	Input	Processing	Output (simulated)
latency-check	{target, count}	Sends simulated probes and computes round-trip stats	{min, avg, max (ms), samples}
traceroute-check	{target, max_hops}	Returns hops from a static lab topology	{hops: [{ttl, addr, rtt_ms}]}
connectivity-check	{target, port}	Attempts a TCP connect to a lab target	{reachable, reason}

Optimization Proposed: Before vs. After

Aspect	BEFORE	AFTER (proposed)
Endpoint location	Hardcoded IP:port in each client config	SVCB record queried at run time (host, port, alpn)
Endpoint change	Edit and redeploy every client	Publisher updates the zone; clients see it after the cache TTL
Registry	Proprietary registry outside DNS	Operator-controlled DNS zone; no Internet-wide registry
Forged DNS data	Accepted without verification	DNSSEC validation; bogus answers (SERVFAIL, no AD) are rejected
Malformed metadata	Undefined behavior or crash	Pinned-subset schema and cap-sha256 check; explicit error
Unavailable service	Hang or generic failure	Bounded timeout and a typed UNAVAILABLE error

Security boundary

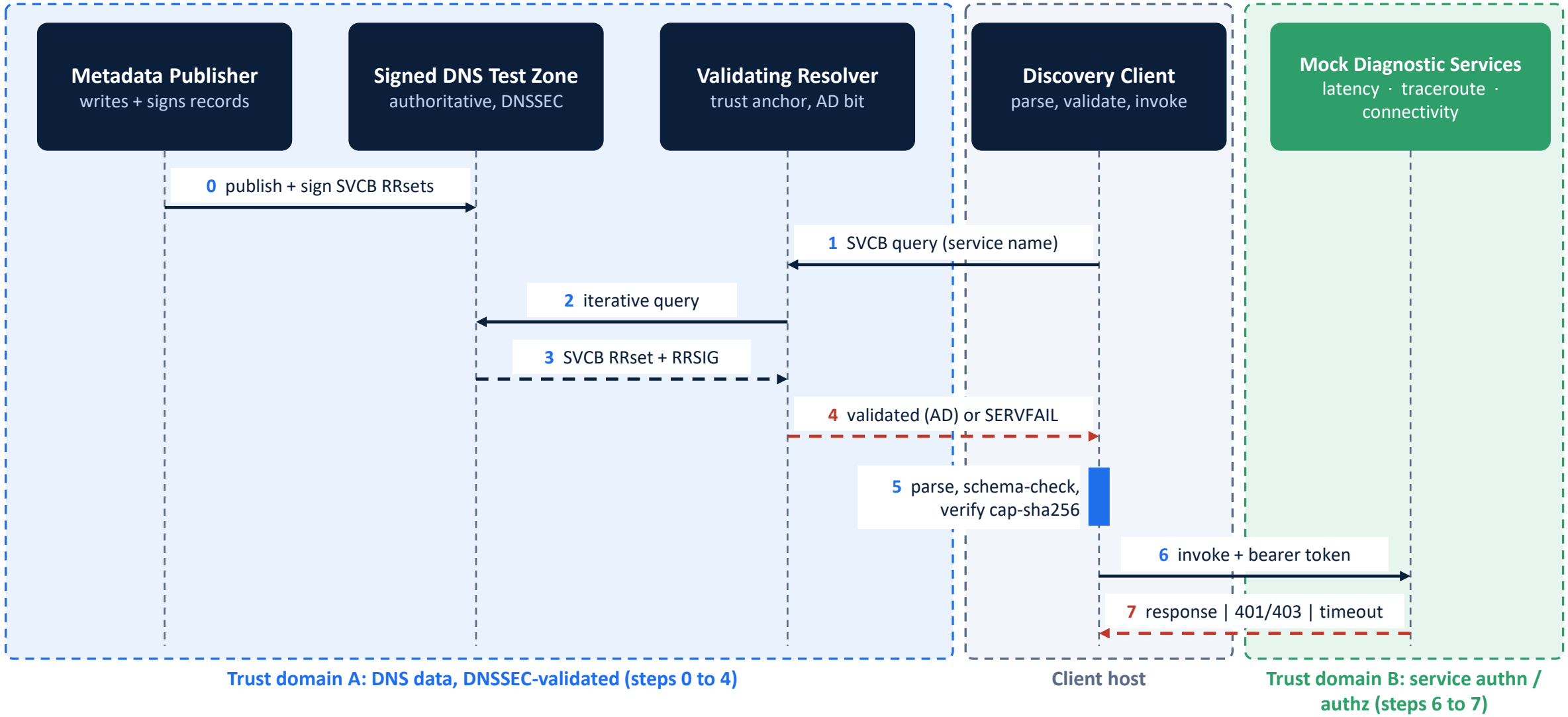
Integrity of metadata
DNSSEC plus schema and digest checks

Authorization
Decided by the service from the caller's token

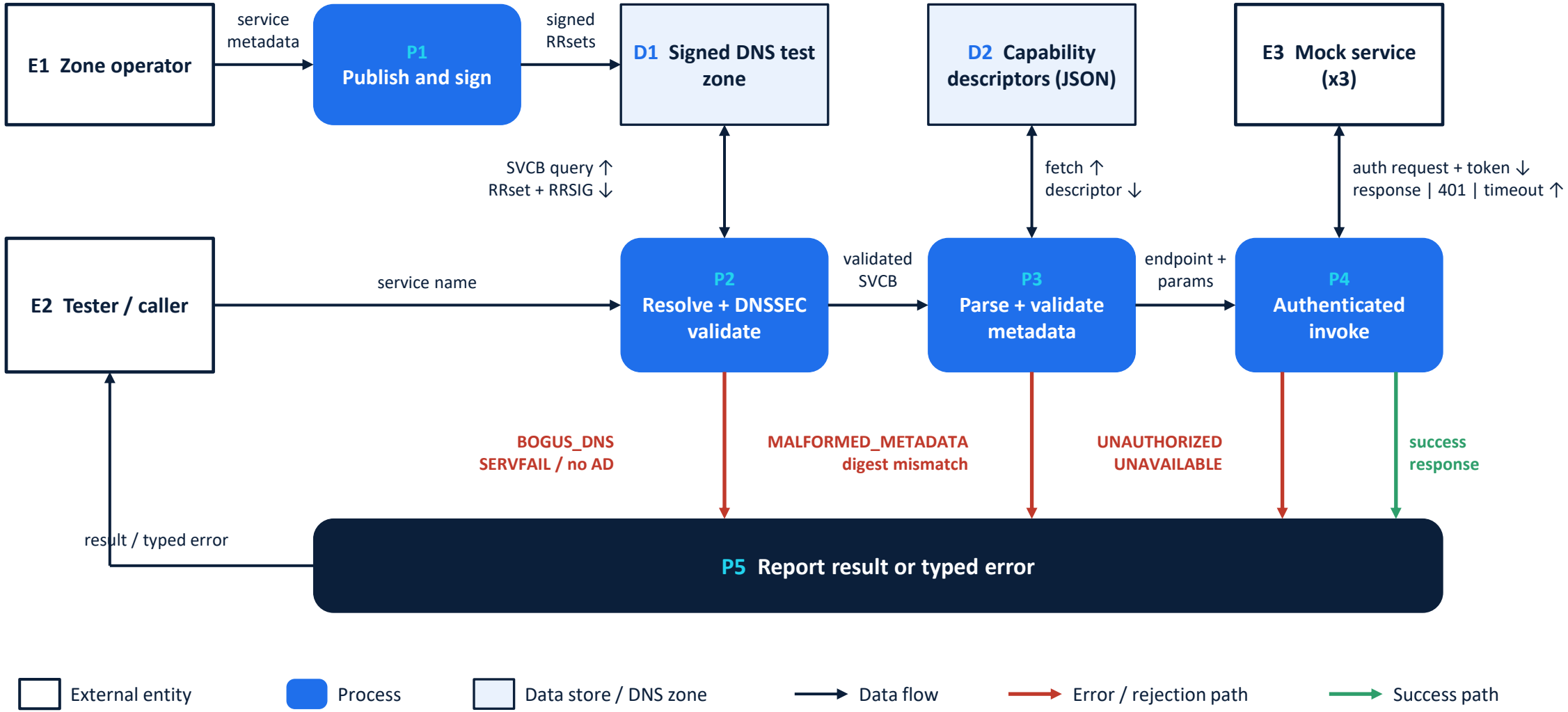
Not proven by either
Whether the service is trustworthy

Proposed improvements only: no latency, speed-up or success-rate figures are claimed; any measurements will be reported separately. Future work: DANE/TLSA endpoint authentication [10].

Solution Architecture & Design



Data Flow Diagram



Deployment Timeline

Phase	Deliverable	Completion criterion	W1	W2	W3	W4
1 **Standards review**	Pinned-subset note	Subset and assumptions agreed				
2 **DNS test-zone setup**	Signed zone + trust anchor	Resolver returns AD for the zone				
3 **Metadata publication**	Publisher + 3 SVCB RRsets	Records load, re-sign, resolve				
4 **Mock service development**	3 HTTP services + descriptors	Each answers a local sample call				
5 **Discovery client**	SVCB query + parser CLI	Prints parsed endpoint per service				
6 **DNSSEC validation**	AD / SERVFAIL handling	Valid accepted, bogus rejected				
7 **Authenticated invocation**	Token check, client + service	200 with token; 401/403 without				
8 **Negative testing**	4-scenario test suite	Each case gives its expected error				
9 **Integration**	End-to-end run script	3 services discovered and invoked				
10 **Documentation and demo**	Report, PDF, pseudocode, demo	Mentor demo runs end to end				

Method: iterative; every phase ends with a runnable check committed to the team's GitHub repo [11]. Durations are relative estimates (W = week), not calendar dates.

Bibliography

[1] J. Mozley, N. Williams, B. Sarikaya, R. Schott, J. Damick, "DNS for AI Discovery," Internet-Draft draft-mozleywilliams-dnsop-dnsaid-02, 27 May 2026 (work in progress). Accessed 11 Oct 2026. https://datatracker.ietf.org/doc/draft-mozleywilliams-dnsop-dnsaid/02/

[2] P. Mockapetris, "Domain Names - Concepts and Facilities," RFC 1034 (STD 13), Nov. 1987. DOI 10.17487/RFC1034. https://www.rfc-editor.org/rfc/rfc1034

[3] P. Mockapetris, "Domain Names - Implementation and Specification," RFC 1035 (STD 13), Nov. 1987. DOI 10.17487/RFC1035. https://www.rfc-editor.org/rfc/rfc1035

[4] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose, "DNS Security Introduction and Requirements," RFC 4033, Mar. 2005. DOI 10.17487/RFC4033. https://www.rfc-editor.org/rfc/rfc4033

[5] R. Arends et al., "Resource Records for the DNS Security Extensions," RFC 4034, Mar. 2005. DOI 10.17487/RFC4034. https://www.rfc-editor.org/rfc/rfc4034

[6] R. Arends et al., "Protocol Modifications for the DNS Security Extensions," RFC 4035, Mar. 2005. DOI 10.17487/RFC4035. https://www.rfc-editor.org/rfc/rfc4035

[7] B. Schwartz, M. Bishop, E. Nygren, "Service Binding and Parameter Specification via the DNS (SVCB and HTTPS Resource Records)," RFC 9460, Nov. 2023. DOI 10.17487/RFC9460. https://www.rfc-editor.org/rfc/rfc9460

[8] P. Hoffman, "DNS Security Extensions (DNSSEC)," BCP 237, RFC 9364, Feb. 2023. DOI 10.17487/RFC9364. https://www.rfc-editor.org/rfc/rfc9364

[9] S. Cheshire, M. Krochmal, "DNS-Based Service Discovery," RFC 6763, Feb. 2013. DOI 10.17487/RFC6763. https://www.rfc-editor.org/rfc/rfc6763

[10] P. Hoffman, J. Schlyter, "The DNS-Based Authentication of Named Entities (DANE) TLS Protocol: TLSA," RFC 6698, Aug. 2012. DOI 10.17487/RFC6698. https://www.rfc-editor.org/rfc/rfc6698

[11] AIORI Hackathon 3, "PS-033: DNS-Based Discovery of Diagnostic Services" and "Proposed-structure-hackathon.pdf" (organizer-provided documents). https://github.com/aiori-hackathon

[12] NLnet Labs, "unbound.conf(5)," Unbound 1.26.1 documentation. https://unbound.docs.nlnetlabs.nl/en/latest/manpages/unbound.conf.html

[13] "dnspython 2.9.0 documentation," Dnspython Contributors. https://dnspython.readthedocs.io/en/stable/

Thank You for Your Attention!

DNS locates the service and DNSSEC authenticates the data;
the service alone decides who may call it.

Questions?

Team Learner